

GEENERA · ALIADOS DEL SOL

Guía integral del Hub

Funcionamiento, decisiones, modelo de datos y puesta en marcha

Versión del 29 de septiembre de 2026 · Fases 1 a 10

Para qué sirve este documento. Reúne en un solo lugar cómo funciona el Hub, por qué se construyó así, las decisiones del equipo, el modelo de datos y los pasos para ponerlo en producción. Si se pierde una conversación con Claude, este archivo y `CLAUDE.md` bastan para retomar el trabajo.

Fecha de corte: 29 de septiembre de 2026. **Estado:** fases 1 a 10 construidas y probadas en el entorno de pruebas (`aliados-dev`). Producción (`aliados-prod`) aún no tiene el esquema ni recibe aliados.

Documentos relacionados: `CLAUDE.md` (contexto técnico permanente, más detallado en reglas e implementación), `docs/HANDOFF.md` y `docs/DESIGN_SYSTEM.md` (diseño del Hub).

Contenido

1. Qué es el Hub, en una página
 2. Las plataformas y qué hace cada una
 3. Preguntas de infraestructura
 4. Planes gratuitos y de pago: ¿hace falta Pro?
 5. Flujos de trabajo (diagramas)
 6. Reglas del programa: puntos, calidad, niveles
 7. Modelo de datos: tablas y columnas
 8. Seguridad y variables de entorno
 9. Decisiones tomadas
 10. Próximos pasos para poner en marcha el Hub
 11. Pendientes y preguntas abiertas
 12. Glosario
-

1. Qué es el Hub, en una página

Aliados del Sol Hub es la plataforma del programa de referidos de GEENERA. Personas y organizaciones (los **aliados**) refieren empresas que podrían instalar energía solar. Por cada avance real de esas empresas en el proceso comercial ganan **Puntos Sol**, suben de **nivel** y pueden **canjear recompensas**.

Lo que puede hacer cada persona hoy:

Quién	Qué hace en el Hub
Visitante	Ve el sitio público, refiere una empresa por el formulario de Clientify ("Refiere tu empresa") y solicita ser aliado.
Aliado pendiente	Se registró y confirmó su correo, pero espera la aprobación de GEENERA. No entra al Hub.
Aliado activo	Entra al Hub: registra oportunidades ("Nueva oportunidad") con factura, sigue el avance de sus referidos, ve su historial de puntos, su nivel, su Racha Solar, hace cursos en la Academy, reporta eventos y ve sus canjes.
Admin (equipo GEENERA)	Usa el panel <code>admin.html</code> : aprueba o rechaza solicitudes, suspende o reactiva cuentas, ajusta puntos, valida eventos, resuelve conflictos con Clientify, administra el catálogo de recompensas y anula canjes. Todo queda auditado.
Proveedor de recompensas (sistema externo)	Consulta el saldo de un aliado por su código y registra canjes con su API key.

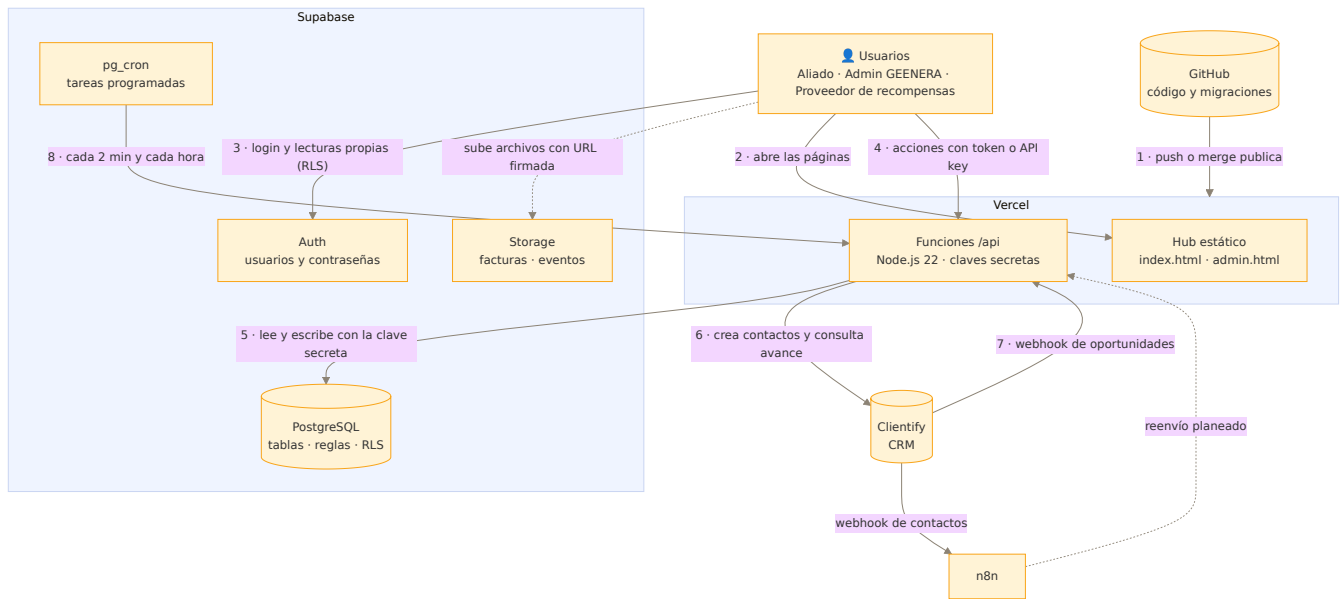
Tres ideas que explican casi todo el diseño:

1. **Cada sistema es dueño de su información.** Supabase es dueño de los puntos y los aliados; Clientify es dueño del proceso comercial; Vercel solo muestra la interfaz y conecta los dos.
2. **Los puntos se escriben en un libro mayor que solo acepta inserciones.** Nunca se edita un saldo a mano: se registran movimientos y la base recalcula todo. Eso da trazabilidad y evita puntos duplicados.
3. **El navegador nunca habla con Clientify ni ve las claves secretas.** Todo lo sensible ocurre en funciones de servidor (`/api`) o dentro de la base de datos, protegido por reglas de seguridad por fila (RLS).

2. Las plataformas y qué hace cada una

Plataforma	Rol en el Hub	Es dueña de	Dónde se configura
GitHub (jmanurodri04-glitch/Alidados-del-sol-hub)	Guarda el código y su historia. Cada fase se trabajó en su propia rama y entra a <code>main</code> por un <i>pull request</i> .	El código fuente, las migraciones de la base y las pruebas.	github.com
Vercel	Publica el sitio (HTML/CSS/JS estático) y ejecuta las funciones de servidor de la carpeta <code>/api</code> (Node.js 22). Cada rama genera un despliegue de prueba (<i>Preview</i>); <code>main</code> genera Producción. También ejecuta un cron diario.	La interfaz y la lógica de integración.	vercel.com → proyecto → Settings
Supabase (dos proyectos: aliados-dev y aliados-prod, región us-east-1)	Base de datos PostgreSQL, autenticación (usuarios y contraseñas), almacenamiento de archivos (facturas y registros de asistentes) y tareas programadas (<code>pg_cron</code>). Aquí viven las reglas de puntos, niveles y seguridad.	Aliados, credenciales, puntos, niveles, racha, módulos, eventos, canjes y la copia de trabajo de las empresas referidas.	supabase.com → proyecto
Clientify (CRM)	Donde el equipo comercial trabaja los leads, las empresas y las oportunidades. El Hub le envía aliados y referidos y lee su avance (Status del contacto, fase de la oportunidad, etiquetas).	El proceso comercial.	Clientify → Configuración
n8n	Automatizaciones existentes de GEENERA. Recibe el webhook de contactos de Clientify (no se cambió). Está planeada una rama que reenvíe al Hub los leads del formulario público.	Sus propios flujos.	Instancia n8n de GEENERA

Diagrama general de la arquitectura



Cómo leerlo: el navegador del aliado solo usa dos cosas públicas: la dirección de Supabase y su *publishable key* (la seguridad la da RLS). Todo lo que requiere permisos especiales (registrar una oportunidad, aprobar un aliado, hablar con Clientify, canjear) pasa por una función de `/api`, que usa la **clave secreta** de Supabase y la **API key** de Clientify guardadas como variables de entorno en Vercel.

Qué hay en el repositorio

Carpeta o archivo	Contenido
<code>index.html</code> y <code>src/Aliados del Sol Hub.dc.html</code>	El Hub (el cuerpo de ambos debe ser idéntico).
<code>admin.html</code> + <code>js/admin.js</code>	Panel de administración.
<code>js/supabase.js</code>	Toda la conexión del Hub con Supabase y con <code>/api</code> (eventos <code>ads:*</code>).
<code>api/</code>	Funciones de servidor (12, ver §8).
<code>lib/</code>	Código compartido de servidor: sesión, cliente de Supabase, integración Clientify (<code>lib/clientify/</code>).
<code>supabase/migrations/</code>	Cambios de la base de datos, en orden. Es la única forma de cambiar el esquema.
<code>supabase/tests/database/</code>	Pruebas pgTAP de las reglas de la base.
<code>tests/</code>	Pruebas de las funciones <code>/api</code> (<code>npm test</code>).
<code>vercel.json</code>	Configuración de Vercel: duración de funciones, cron diario y encabezados.
<code>assets/legal/</code>	Términos y Política de datos en PDF, con la fecha de versión en el nombre.

3. Preguntas de infraestructura

3.1 ¿Por qué hay dos bases de datos, aliados-dev y aliados-prod?

Porque **probar sobre datos reales es peligroso** y en este sistema hay cosas que no se pueden deshacer fácilmente:

- **El libro de puntos solo acepta inserciones.** Un error en una prueba (por ejemplo, otorgar +150 a un aliado real) queda registrado para siempre y solo se corrige con otro movimiento.
- **Las migraciones cambian la estructura.** Una migración con un error puede bloquear el registro o borrar datos. En `dev` se aplica primero, se prueba con pgTAP y con el Preview de Vercel, y solo después se aplica igual en `prod`.
- **Hay datos personales (Ley 1581).** Los datos de prueba no deben mezclarse con los de aliados y empresas reales.
- **Los procesos automáticos se ejecutan solos.** Los cron jobs y los webhooks trabajan cada 2 minutos. En `dev` pueden fallar sin afectar a nadie.

Regla de trabajo: el conector de Supabase de Claude apunta **solo a aliados-dev** (ref `aqqlagfvanuokhmfyynv`). `aliados-prod` (ref `pysyxcrybayhrescplc`) nunca se toca desde las sesiones de desarrollo: se actualiza aplicando las mismas migraciones del repositorio, de forma controlada (ver §10).

Clientify es uno solo para pruebas y producción. Por eso, fuera de Producción el Hub solo envía a Clientify correos que contengan `+prueba` y les pone la etiqueta `PRUEBA HUB`, para poder identificarlos y borrarlos.

3.2 ¿Por qué Vercel tiene variables de *Production*, *Preview* y *Development*?

Vercel crea un **despliegue distinto según el origen del código**, y cada uno puede usar valores diferentes de las mismas variables:

Entorno de Vercel	Cuándo se crea	A qué base apunta	Para qué sirve
Production	Al fusionar en <code>main</code> (o al promover un despliegue). Responde en el dominio oficial.	<code>aliados-prod</code>	Los aliados reales.
Preview	En cada <i>push</i> a cualquier otra rama (p. ej. <code>fase-10-canjes</code>). Tiene una URL propia por rama y por despliegue.	<code>aliados-dev</code>	Probar una funcionalidad antes de fusionarla, sin tocar producción.
Development	Cuando un desarrollador ejecuta <code>vercel dev</code> en su computador (<code>vercel env pull</code> descarga estas variables).	<code>aliados-dev</code>	Desarrollo local.

Como el sitio es estático y no tiene paso de *build*, el navegador **no puede leer variables de entorno**. Por eso existe `GET /api/config`, que devuelve la URL y la *publishable key* del entorno donde corre. Así el mismo código sirve para Preview (dev) y Production (prod) sin cambiar nada.

Consecuencia práctica: la variable `SUPABASE_URL`, por ejemplo, tiene dos valores en Vercel: el de `aliados-prod` marcado solo para *Production* y el de `aliados-dev` marcado para *Preview* y *Development*.

3.3 ¿Por qué ramas por fase y *pull requests*?

- Cada fase (1 a 10) se construyó en su propia rama (`fase-1-base-datos` ... `fase-10-canjes`). Cada rama se construyó sobre la anterior, así que `fase-10-canjes` **contiene todo**.
- Una rama genera un Preview en Vercel que apunta a `dev` : se puede probar la fase completa con un enlace, sin afectar producción.
- El *pull request* hacia `main` es el punto de control: se revisa qué cambia, y al fusionarlo Vercel publica Producción.
- **Importante:** fusionar en `main` publica el **código**, pero **no** cambia la base de `prod`. Las migraciones de la base se aplican aparte (§10, paso 3). Por eso el orden de puesta en marcha importa.

3.4 ¿Por qué parte de la lógica está en la base de datos y no en el código de Vercel?

Las reglas críticas (puntos, niveles, racha, límites, validaciones de admin) están en **funciones SQL dentro de Supabase**, no solo en JavaScript, porque:

- **Se ejecutan en una transacción:** o se hace todo (insertar el canje y descontar los puntos) o nada.
- **Bloquean la fila del aliado:** dos operaciones simultáneas no pueden gastar el mismo saldo.
- **No dependen de quién llama:** el Hub, el panel, un webhook o un cron pasan por las mismas reglas.
- **Se prueban con pgTAP** (unas 500 verificaciones entre todas las fases).

Las funciones de Vercel se encargan de lo que la base no puede hacer: verificar el token de sesión, hablar con Clientify, generar URLs firmadas y responder al navegador.

4. Planes gratuitos y de pago: ¿hace falta Pro?

Datos tomados de la documentación oficial de Vercel y Supabase en septiembre de 2026. Los precios están en dólares y pueden cambiar: confírmalos en vercel.com/pricing y supabase.com/pricing antes de pagar.

4.1 Vercel: Hobby frente a Pro

Aspecto	Hobby (gratis)	Pro (USD 20 por usuario/mes, incluye USD 20 de consumo)
Uso permitido	Solo personal y no comercial.	Uso comercial.
Cron jobs de Vercel	Máximo 1 ejecución al día por cron.	Hasta cada minuto.
Funciones	El equipo trabaja con el tope de 12 funciones por despliegue ; hoy el Hub usa exactamente 12.	Sin ese tope práctico.
Duración máxima de una función	Menor.	Mayor (el Hub pide hasta 60 s).
Volver a un despliegue anterior (<i>rollback</i>)	Limitado.	Sí, con un clic o <code>vercel rollback</code> .
Equipo	Una sola persona.	Varios miembros con roles.
Protección de despliegues y soporte	Básico.	Más opciones y soporte.

Recomendación: pasar Vercel a Pro antes de abrir el programa al público.

1. **Condiciones de uso.** El Hub es de una empresa (GEENERA) y es parte de su operación comercial. El plan Hobby solo admite uso personal y no comercial, así que Producción en Hobby incumpliría las condiciones de Vercel.
2. **Límite de funciones.** Con 12 funciones el Hub llegó al tope de Hobby: cualquier endpoint nuevo exigiría unir funciones.
3. **Operación.** Poder revertir un despliegue malo en segundos y tener a varias personas de GEENERA con acceso es importante en producción.

Los cron frecuentes **no** dependen de Vercel Pro: el Hub los ejecuta desde Supabase (`pg_cron`), así que funcionan en cualquier plan de Vercel. El cron diario de Vercel es solo un respaldo.

Hoy la cuenta está en **prueba de Pro** (quedaban 6 días al 29 de septiembre). Si la prueba termina sin pagar, la cuenta vuelve a Hobby: el sitio sigue funcionando mientras no se superen los límites, pero se pierde lo anterior.

4.2 Supabase: Free frente a Pro

Aspecto	Free (gratis)	Pro (USD 25/mes por organización + cómputo)
Pausa por inactividad	Se pausa si durante 7 días no hay suficiente actividad. Avisa por correo; se reactiva a mano.	Nunca se pausa.
Copias de seguridad	No hay copias descargables. Se recomienda exportar a mano (<code>supabase db dump</code>).	Copia diaria automática, 7 días de historial y restauración desde el panel.
Proyectos	2 proyectos gratis en total.	Ilimitados; cada proyecto paga su cómputo.
Tamaño de la base	500 MB por proyecto.	8 GB incluidos por proyecto.
Archivos (Storage)	1 GB.	100 GB.
Usuarios activos al mes	50 000.	100 000.
Transferencia (egress)	5 GB.	250 GB.
Soporte	Comunidad.	Soporte por ticket.
Cómputo	Nano incluido.	USD 10 de crédito al mes, que cubre un proyecto Micro (~USD 10). Cada proyecto adicional suma su cómputo.

Recomendación: pasar a Pro solo la organización de aliados-prod, cuando se abra el registro a aliados reales. Motivos:

1. **La pausa sería una caída del programa.** Si en una semana no hay suficiente actividad (algo probable al inicio), Supabase pausa el proyecto: el Hub, el login y la sincronización con Clientify dejan de funcionar hasta que alguien lo reactive.
2. **Las copias de seguridad son indispensables.** El libro de puntos y los datos de aliados son registros del programa. En Free no hay copia automática descargable.
3. **Los límites de Free alcanzan para empezar** (500 MB y 1 GB de facturas dan para cientos de aliados), así que el motivo de pagar es la **disponibilidad y la seguridad**, no el espacio.

Cómo hacerlo sin pagar dos veces: Supabase cobra por **organización** y no mezcla planes dentro de una organización. Se puede dejar `aliados-dev` en una organización gratuita y `aliados-prod` en una organización Pro. Así el costo es de unos **USD 25/mes** (el crédito de cómputo cubre un proyecto Micro). Si ambos proyectos quedaran en la misma organización Pro, se pagaría además el cómputo de `dev` (~USD 10/mes).

Mientras siga en Free: entrar al panel de Supabase al menos una vez por semana, responder los correos de aviso de pausa y exportar la base periódicamente.

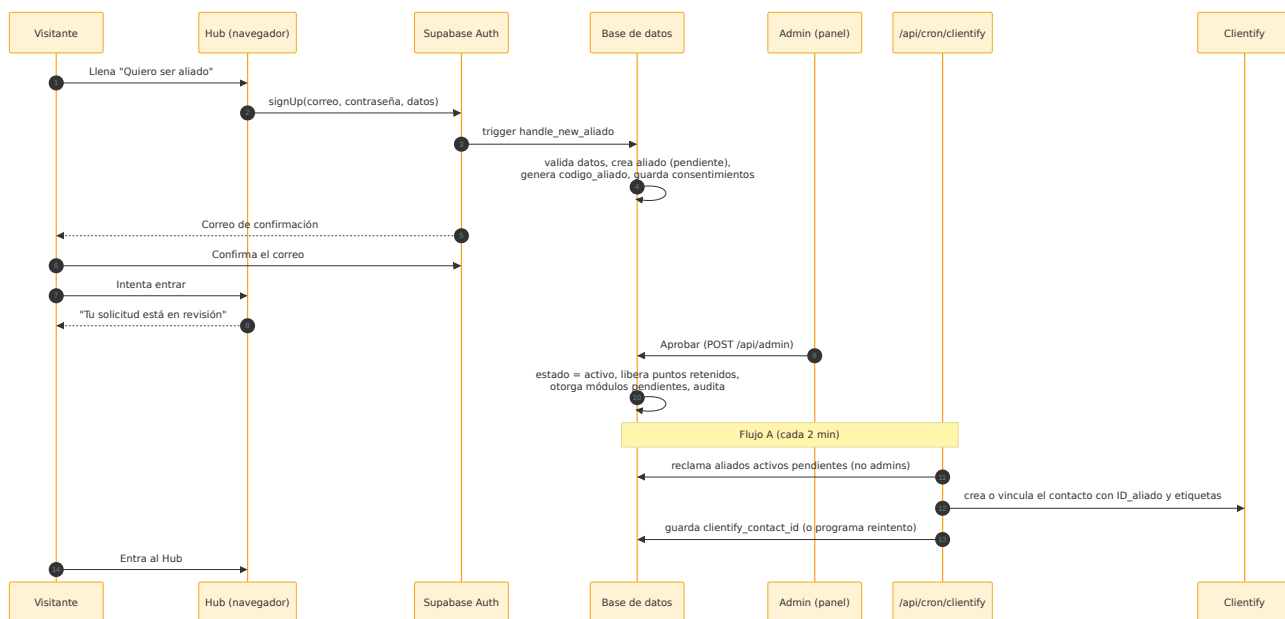
4.3 Costo mensual estimado para arrancar

Concepto	Costo aproximado (USD/mes)
Vercel Pro (1 usuario)	20

Concepto	Costo aproximado (USD/mes)
Supabase Pro (organización de prod , 1 proyecto Micro)	25
Supabase Free (organización de dev)	0
Servicio de correo SMTP (p. ej. Resend, Brevo, SES; ver §10, paso 4)	0-20 según volumen
Total aproximado	45-65

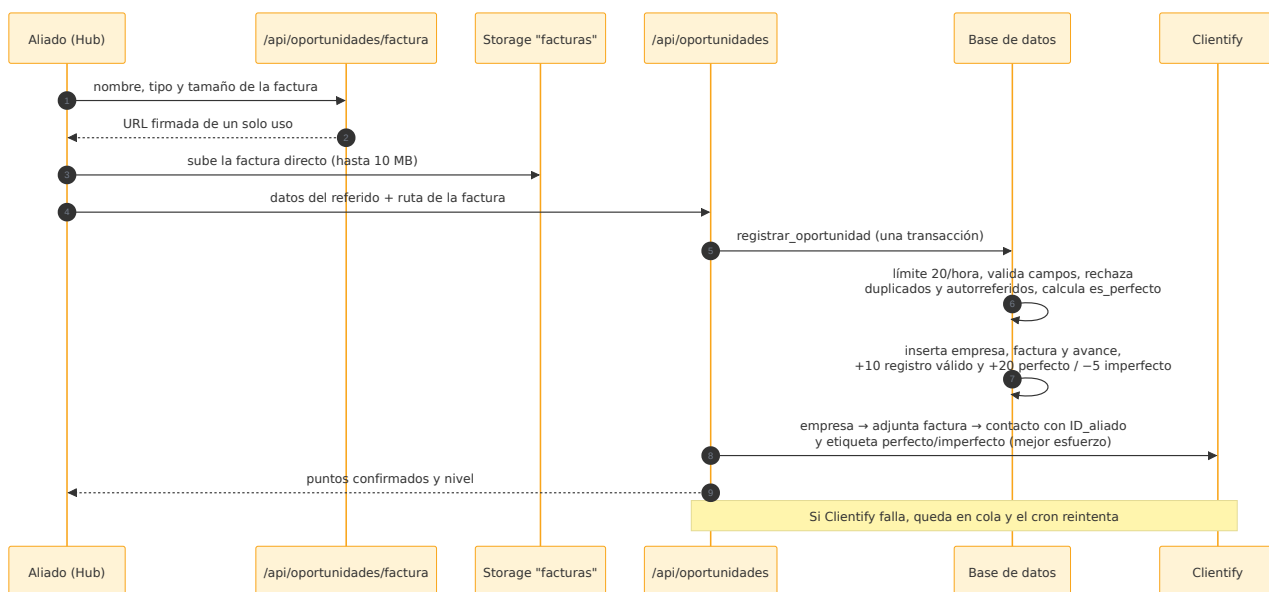
5. Flujos de trabajo (diagramas)

5.1 Registro y aprobación de un aliado

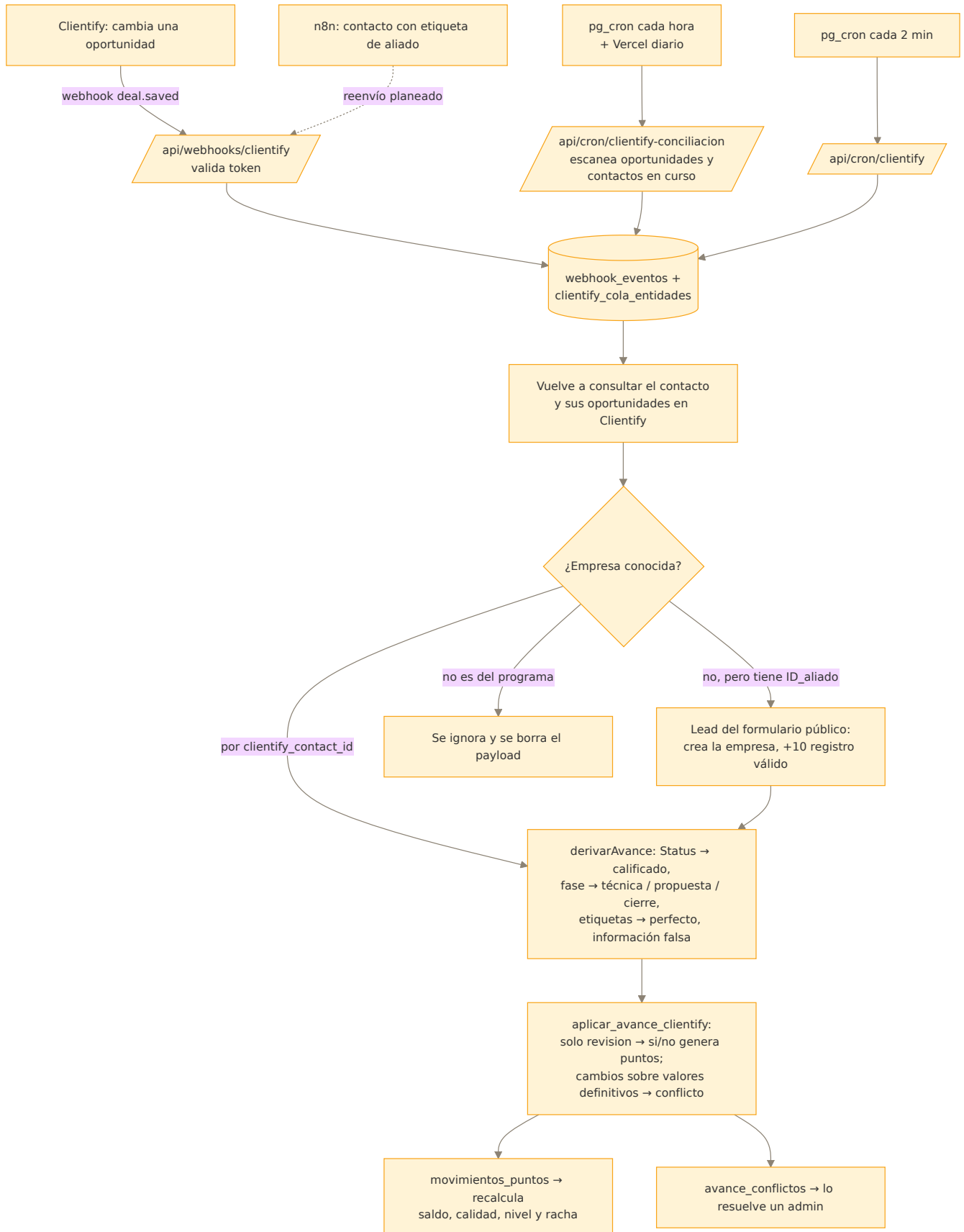


Por qué así: la contraseña la maneja solo Supabase Auth (nunca se guarda en las tablas). El aliado no se envía a Clientify al registrarse sino al aprobarse, para no llenar el CRM de solicitudes que se rechazarán. Si Clientify falla, ni el registro ni la aprobación fallan: queda en cola y se reintenta.

5.2 Nueva oportunidad desde el Hub (flujo B)



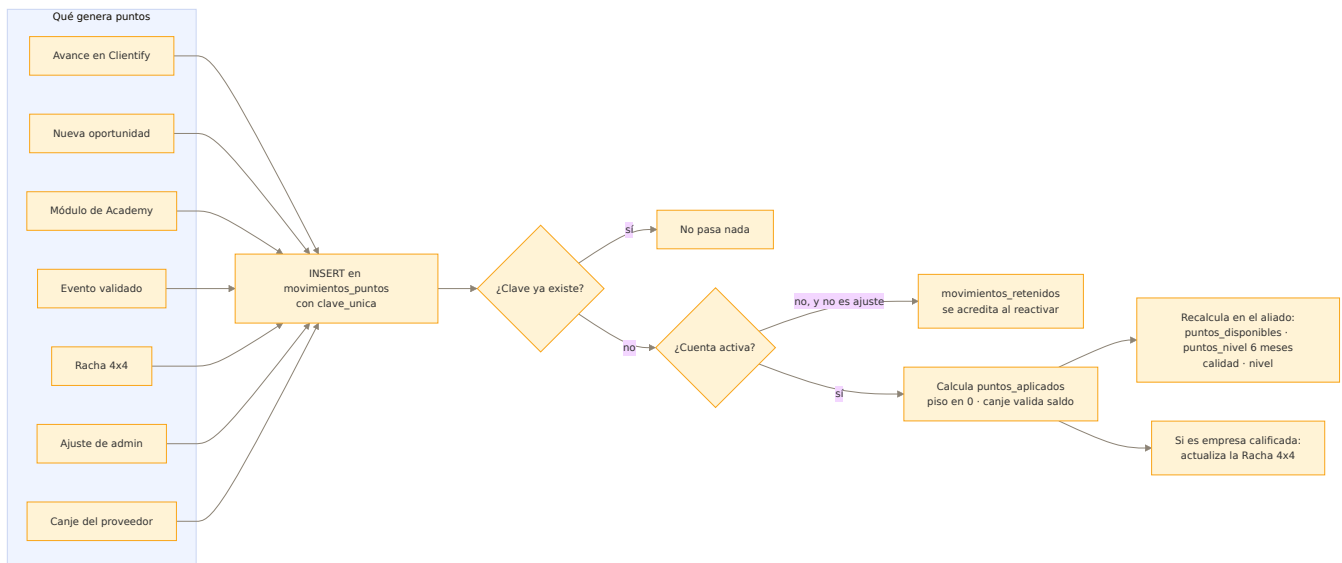
5.3 Avance comercial desde Clientify (flujo C) y conciliación



Por qué así:

- **Se vuelve a consultar Clientify** en vez de confiar en el webhook: el webhook puede llegar incompleto, repetido o desordenado.
- **La cola agrupa eventos:** si llegan cinco eventos de la misma oportunidad, se procesa una sola vez.
- **La conciliación horaria** cubre lo que no llega por webhook. El webhook de contactos va a n8n, así que los cambios de Status llegan como máximo con una hora de retraso.
- **Cada variable da puntos una sola vez en toda su vida** (clave única). Por eso reprocesar es inofensivo.

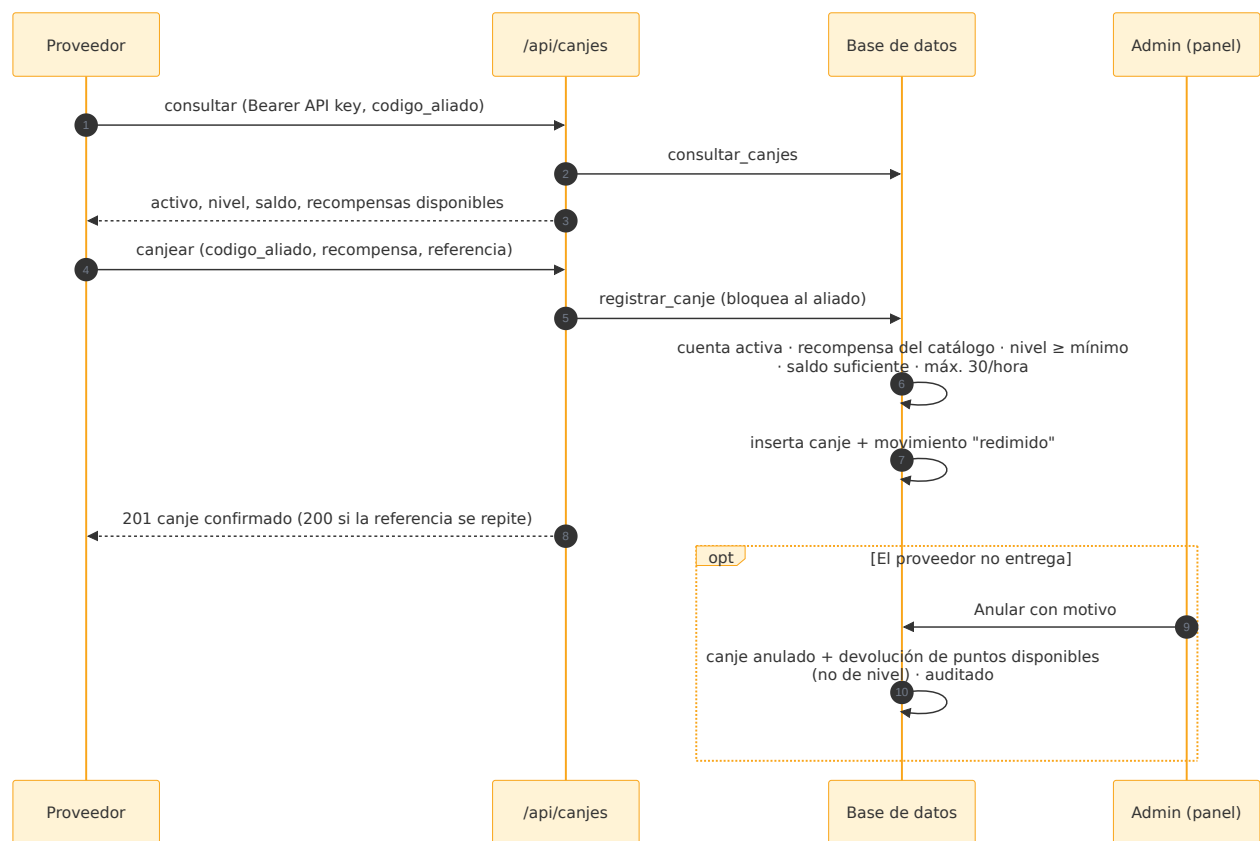
5.4 Cómo se mueven los puntos



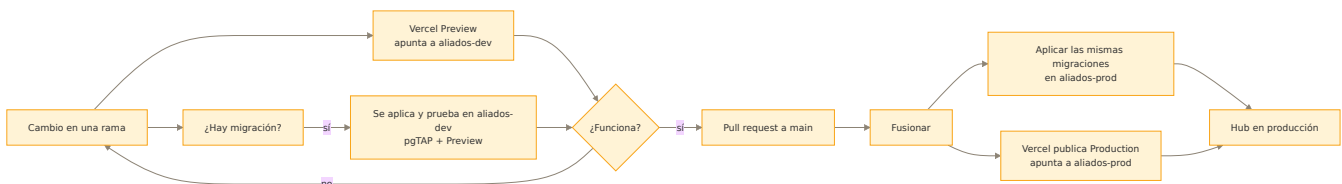
5.5 Eventos aliados

1. El aliado reporta el evento en Puntos Sol → "Reporta un evento", con archivo de asistentes opcional (URL firmada al bucket privado `eventos`). Máximo 5 al día.
2. Queda `pendiente`. En el panel, un admin ve si cumple las 4 condiciones: evento realizado, GEENERA involucrada, registro de asistentes y al menos 5 empresas dentro del perfil.
3. **Validar** otorga +100 una sola vez (clave `evento:{id}`). **Rechazar** exige un motivo, que el aliado ve.

5.6 Canjes



5.7 De un cambio en el código a producción



Orden importante: si el código nuevo usa una tabla o función nueva, la migración en `prod` debe aplicarse **antes o al mismo tiempo** que la publicación. Si no, Producción mostrará errores hasta que se aplique.

5.8 Tareas programadas

Tarea	Dónde	Frecuencia (hora Bogotá)	Qué hace
sincronizar-clientify-aliados	Supabase <code>pg_cron</code> → <code>/api/cron/clientify</code>	Cada 2 min	Procesa las colas: aliados aprobados (flujo A), oportunidades (flujo B) y eventos de Clientify (flujo C).
conciliar-clientify	Supabase <code>pg_cron</code> → <code>/api/cron/clientify-conciliacion</code>	Cada hora, minuto 7	Escanea oportunidades y contactos en curso para cubrir webhooks perdidos.
Conciliación de respaldo	Vercel Cron (<code>vercel.json</code>)	Diario 02:00	Lo mismo, como respaldo si <code>pg_cron</code> falla.

Tarea	Dónde	Frecuencia (hora Bogotá)	Qué hace
recalcular-puntos-diario	Supabase pg_cron	Diario 00:15	Recalcula saldos y niveles (los puntos de nivel vencen a los 6 meses).
reiniciar-rachas-semanal	Supabase pg_cron	Lunes 00:05	Reinicia rachas completadas o interrumpidas.
otorgar-modulos-mensual	Supabase pg_cron	Día 1, 00:05	Otorga módulos pendientes dentro del tope de 20 puntos del mes.
depurar-webhooks-clientify	Supabase pg_cron	Día 1, 03:30	Vacía los payloads de webhooks de más de 90 días (datos personales).

Los cron de Supabase que llaman a Vercel necesitan tres secretos en el **Vault** de cada proyecto: `clientify_sync_url` (URL del despliegue), `cron_secret` (igual a `CRON_SECRET` de Vercel) y, si el despliegue está protegido, `vercel_bypass_secret`. Sin ellos, el job no hace nada.

6. Reglas del programa: puntos, calidad, niveles

6.1 Tabla de Puntos Sol

Qué pasa	Puntos	Se otorga...
Registro válido de un referido	+10	Una vez por empresa
Referido perfecto (10 campos + factura)	+20	Una vez por empresa (comparte clave con imperfecto)
Referido imperfecto	-5	Idem
Empresa calificada (Status "lead caliente" o superior)	+30	Una vez (comparte clave con no calificado)
Referido no calificado	-10	Idem
Fuera del perfil (no calificado y perfecto)	-15	Se suma al -10: -25 en total
Evaluación técnica (fase "Diseño" o posterior)	+30	Una vez
Propuesta comercial (fase "Presentación de oferta" o posterior)	+50	Una vez
Negocio cerrado (fase "Contrato")	+150	Una vez
Información falsa (etiquetas "fraude", "no existe", "información de contacto errónea")	-30	Una vez
Baja calidad reiterada (la registra un admin)	-20	Máximo una por aliado y día
Módulo de Academy completado	Según el módulo (hoy 5 o 0)	Máximo 20 puntos por mes; lo que no cabe espera al mes siguiente
Evento aliado validado	+100	Una vez por evento
Racha Solar 4x4 completada	+75	Máximo una cada 28 días
Ajuste de admin	±	Con justificación
Canje	- (valor de la recompensa)	Solo con saldo suficiente

Reglas generales:

- **Nada genera puntos dos veces.** Cada evento tiene una clave única. Si Clientify cambia después un valor definitivo (por ejemplo, de "no calificado" a "calificado"), no se dan puntos automáticamente: queda un **conflicto** para un admin.
- **"En revisión" nunca da ni quita puntos.**
- **Piso en 0 y sin memoria.** El saldo nunca es negativo. Si una penalización es mayor que el saldo, se descuenta lo que haya y el resto se pierde (no queda deuda).
- **Dos saldos:**
 - `puntos_disponibles`: lo que se puede canjear. Es histórico y no vence.
 - `puntos_nivel`: ganados menos perdidos de los **últimos 6 meses**, con piso en 0. Sirve para el nivel. Un canje no los descuenta y la devolución de un canje anulado tampoco los suma.
- **Cuenta no activa:** si el aliado está pendiente o suspendido, sus puntos quedan **retenidos** y se acreditan al activarse. Los ajustes de admin se aplican siempre.

6.2 Calidad de referidos

```
calidad_empresa = 100 × (0,40·calificado + 0,30·perfecto + 0,20·evaluación técnica + 0,10·integridad)
```

Cada variable vale 1 si es "sí" y 0 si es "no". Si **alguna** está "en revisión", la empresa no cuenta todavía (para no castigar lo que está en proceso). La calidad del aliado es el promedio de sus empresas evaluables, sin ventana de tiempo. El *lead scoring* de Clientify **no** se usa para la calidad.

6.3 Niveles

Nivel	Puntos de nivel ≥	Calidad ≥
Círculo Solar	1000	85 %
Diamante	700	80 %
Platino	450	70 %
Oro	250	60 %
Plata	100	50 %
Bronce	0	—

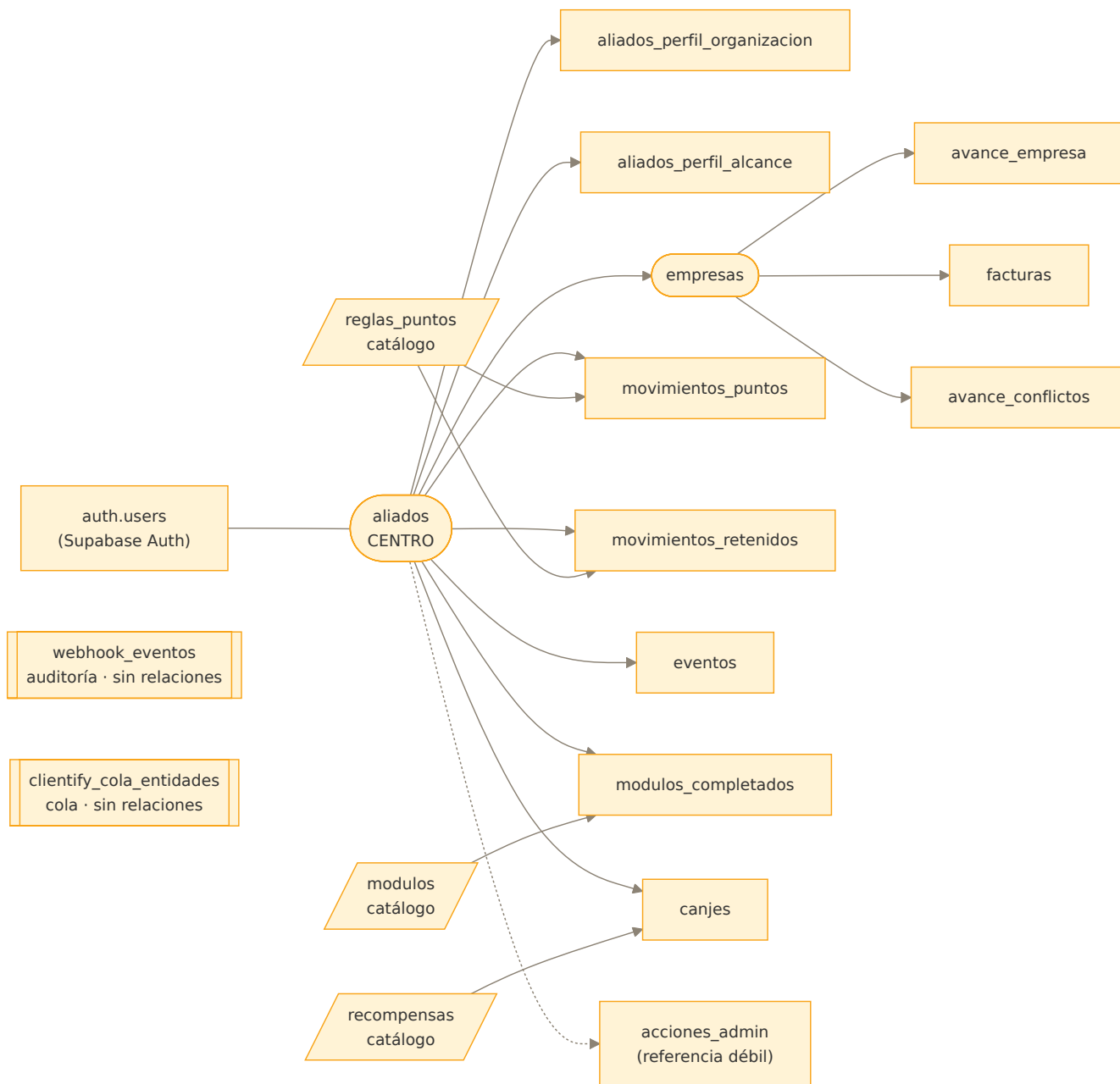
El nivel es **el menor** entre el que dan los puntos y el que da la calidad. Ejemplo: 1000 puntos con 60 % de calidad = Oro. El Hub no muestra los requisitos de calidad, pero la regla sí se aplica.

6.4 Racha Solar 4x4

Una semana (lunes a domingo, hora Bogotá) cuenta si en ella **se calificó** al menos una empresa del aliado. Cuatro semanas seguidas = +75. La racha se ve completa el resto de esa semana y se reinicia el lunes siguiente; una semana sin calificaciones la reinicia.

7. Modelo de datos: tablas y columnas

7.1 Por qué unas tablas se relacionan con casi todo y otras con nada



Flecha continua: la tabla de la derecha guarda una llave foránea hacia la de la izquierda. Flecha punteada: referencia débil (se vacía si se borra la cuenta). Paralelogramos: catálogos. Doble borde: tablas independientes a propósito.

aliados es el centro porque todo en el programa le pertenece a un aliado: sus empresas, sus puntos, sus eventos, sus módulos, sus canjes. Por eso casi todas las tablas tienen una columna `aliado_id` que apunta a `aliados.id`. Esa relación permite:

- **Seguridad:** la regla RLS "cada aliado solo lee sus filas" se escribe como `aliado_id = auth.uid()`.

- **Borrado completo:** si una persona pide eliminar su cuenta (Ley 1581), al borrar el usuario se borra en cascada todo lo suyo.
- **Cálculos:** los saldos, la calidad y el nivel se recalculan filtrando por `aliado_id`.

empresas es el segundo centro: cada referido tiene su avance (`avance_empresa`), su factura (`facturas`) y sus conflictos con Clientify (`avance_conflictos`).

Tablas sin relaciones, a propósito:

Tabla	Por qué no se relaciona
<code>webhook_eventos</code>	Es un registro de auditoría de lo que llegó de Clientify. Se guarda tal cual, aunque el contacto no sea del programa o todavía no exista en la base. Si tuviera llaves foráneas, un evento de una empresa aún desconocida no se podría guardar, y se perdería.
<code>clientifyColaEntidades</code>	Es una cola de trabajo identificada por los IDs de Clientify (<code>entidad</code> , <code>entidad_id</code>), no por IDs internos. Se vacía a medida que se procesa.
<code>reglas_puntos</code> , <code>modulos</code> , <code>recompensas</code>	Son catálogos : no apuntan a nadie, pero otras tablas apuntan a ellos (<code>movimientos_puntos.motivo</code> , <code>modulos_completados.modulo_id</code> , <code>canjes.recompensa_id</code>). Sus filas no se pueden borrar si están en uso.
<code>acciones_admin</code>	Se relaciona con <code>aliados</code> de forma débil : si se borra la cuenta, la referencia queda vacía pero se conserva el código en texto (<code>admin_codigo</code> , <code>aliado_codigo</code>), para que la auditoría no se pierda.

Por qué `movimientos_puntos.vinculo_id` es texto y no una llave foránea: un movimiento puede venir de una empresa, un evento, un módulo, una racha, un canje o un ajuste. Una sola columna no puede apuntar a seis tablas distintas. Por eso se guardan dos datos, `vinculo` (qué tabla) y `vinculo_id` (qué fila), y las vistas unen según el caso.

Llaves compuestas entre `empresas` y `facturas`: la factura apunta a (`empresa_id` , `aliado_id`) de la empresa, y la empresa apunta a (`factura_id` , `id`) de la factura. Así la base garantiza que una factura no pueda quedar asociada a la empresa de otro aliado.

7.2 Estados y tipos que se repiten

Tipo	Valores	Uso
<code>estado_triple</code>	<code>si</code> , <code>no</code> , <code>revision</code>	Variables de avance y calidad: "revision" = aún no se sabe.
<code>tipo_aliado</code>	<code>financiero</code> , <code>emi</code> , <code>linker</code> , <code>cliente_embajador</code> , <code>agremiaciones</code>	Define el formulario, el perfil y el dashboard.
<code>nivel</code>	<code>bronce</code> , <code>plata</code> , <code>oro</code> , <code>platino</code> , <code>diamante</code> , <code>circulo_solar</code>	Nivel del aliado y nivel mínimo de una recompensa (el orden importa: se comparan).
<code>aliados.estado</code>	<code>pendiente</code> , <code>activo</code> , <code>suspendido</code> , <code>rechazado</code>	Solo <code>activo</code> entra al Hub y gana puntos.
<code>clientify_sync_estado</code>	<code>pendiente</code> , <code>ok</code> , <code>error</code> , <code>excluido</code>	Estado del envío a Clientify; <code>excluido</code> = cuenta de admin.

7.3 Tablas del aliado

aliados — el aliado y sus valores calculados

Columna	Qué representa
id	Llave técnica, igual al usuario de Supabase Auth. Nunca se muestra ni sale del sistema.
codigo_aliado	Código público (p. ej. LKCAL5SCPTB9T): prefijo del tipo + iniciales + 8 caracteres. Es el ID_aliado de Clientify. Inmutable.
nombre_completo, correo, celular, regional	Datos del registro. El correo es único; el celular en formato internacional (+57...).
tipo_aliado	Tipo de aliado (§7.2).
puntos_nivel	Puntos de los últimos 6 meses, con piso en 0. Define el nivel. Calculado.
puntos_disponibles	Saldo canjeable, histórico. Calculado.
calidad_referidos	Promedio de calidad de sus empresas evaluables (0-100) o vacío. Calculado.
nivel	Nivel vigente. Calculado.
racha_semana_1..4, racha_ultima_semana	Estado de la Racha Solar y el lunes de la última semana contada.
clientify_contact_id	ID del contacto del aliado en Clientify.
clientify_sync_estado, clientify_sync_error, clientify_sync_intentos, clientify_sync_proximo_at, clientify_sync_at	Cola del flujo A: estado, último error, reintentos, cuándo reintentar y última sincronización exitosa.
autorizacion_datos_at, terminos_aceptados_at	Cuándo autorizó el tratamiento de datos y aceptó los términos (Ley 1581).
terminos_version, politica_datos_version	Qué versión de cada documento aceptó.
estado	pendiente, activo, suspendido, rechazado.
aprobado_at, aprobado_por	Cuándo y qué admin aprobó la solicitud.
rol	aliado o admin. Nunca se toma del navegador.
created_at, updated_at	Fechas de creación y de última modificación.

aliados_perfil_organizacion — Financieros y Agremiaciones (1 a 1)

Columna	Qué representa
aliado_id	El aliado (llave primaria y foránea).
organizacion, cargo	Entidad y cargo del aliado.
created_at, updated_at	Fechas.

aliados_perfil_alcance — EMI, Linker y Cliente Embajador (1 a 1)

Columna	Qué representa
aliado_id	El aliado.

Columna	Qué representa
<code>como_llega_empresas</code>	Respuesta a "¿Cómo llegas a las empresas?".
<code>created_at</code> , <code>updated_at</code>	Fechas.

Por qué dos tablas de perfil y no columnas en `aliados`: cada tipo pide datos distintos. Separarlos evita columnas vacías y permite que la base valide que el perfil corresponda al tipo.

7.4 Referidos

`empresas` — cada referido

Columna	Qué representa
<code>id</code>	Llave interna de la empresa referida.
<code>aliado_id</code>	Aliado que la refirió (siempre tomado de la sesión, nunca del formulario).
<code>origen</code>	<code>hub</code> (Nueva oportunidad) o <code>clientify_form</code> (formulario público).
<code>empresa</code> , <code>sector</code> , <code>subsector</code> , <code>ciudad</code>	Datos de la empresa. Obligatorios desde el Hub; pueden faltar si vienen del formulario público.
<code>nombre_contacto</code> , <code>cargo</code> , <code>telefono</code> , <code>correo</code>	Contacto en la empresa. El correo es único en todo el programa: gana el primer aliado.
<code>valor_factura</code>	Valor mensual de la factura de energía (COP).
<code>observaciones</code>	Comentarios libres (no cuentan para "perfecto").
<code>factura_id</code>	Factura adjunta, si hay.
<code>es_perfecto</code>	Si se registró con los 10 campos + factura. Se calcula al guardar.
<code>clientify_contact_id</code> , <code>clientify_company_id</code> , <code>clientify_deal_id</code>	IDs del contacto, la empresa y la oportunidad en Clientify.
<code>clientify_sync_estado</code> , <code>clientify_sync_error</code> , <code>clientify_sync_intentos</code> , <code>clientify_sync_proximo_at</code> , <code>clientify_sync_at</code>	Cola del flujo B (envío a Clientify).
<code>autorizacion_contacto_at</code>	Cuándo el aliado declaró tener autorización del contacto (Ley 1581). Obligatoria desde el Hub.
<code>created_at</code> , <code>updated_at</code>	Fechas.

`facturas` — factura adjunta a un referido

Columna	Qué representa
<code>id</code>	Llave de la factura.

Columna	Qué representa
<code>empresa_id</code> , <code>aliado_id</code>	Empresa y aliado dueños (llave compuesta con <code>empresas</code>).
<code>tipo_documento</code>	<code>pdf</code> , <code>jpg</code> o <code>png</code> .
<code>storage_path</code>	Ruta en el bucket privado <code>facturas: {aliado}/{empresa}/{archivo}</code> .
<code>nombre_archivo</code>	Nombre original del archivo.
<code>fecha_carga</code>	Cuándo se subió.
<code>validacion</code> , <code>aprobado</code>	Revisión de la factura por GEENERA (<code>pendiente</code> / <code>revisado</code> ; <code>si</code> / <code>no</code> / <code>revision</code>).
<code>clientify_subida_at</code>	Cuándo se adjuntó a la empresa en Clientify.

avance_empresa — espejo del avance en Clientify (1 a 1 con `empresas`)

Columna	Qué representa
<code>empresa_id</code>	La empresa.
<code>calificado</code>	Del Status del contacto (<code>caliente</code> o <code>superior</code> = sí; <code>no calificado</code> = no).
<code>perfecto</code>	Inicia con <code>es_perfecto</code> ; Clientify puede corregirlo con la etiqueta.
<code>oportunidad_tecnica</code>	Evaluación técnica: la oportunidad llegó a "Diseño".
<code>integridad_informacion</code>	Sigue a <code>calificado</code> . Solo cuenta para la calidad.
<code>calidad_empresa</code>	Calidad ponderada (0-100) o vacía si algo sigue en revisión. Calculada.
<code>propuesta_comercial</code> , <code>negocio_cerrado</code>	La oportunidad llegó a "Presentación de oferta" / "Contrato".
<code>informacion_falsa</code>	Etiquetas de fraude en Clientify.
<code>fuera_perfil</code>	No calificado y perfecto.
<code>estado_contacto_clientify</code>	Status crudo del contacto (p. ej. <code>hot-lead</code>).
<code>fase_oportunidad</code> , <code>fase_oportunidad_num</code>	Fase cruda de la oportunidad y su número.
<code>estado_oportunidad</code>	<code>abierta</code> , <code>ganada</code> o <code>perdida</code> .
<code>lead_scoring</code>	Solo informativo (la API actual no lo entrega).
<code>valor_oportunidad</code>	Pipeline originado: suma del importe de oportunidades ganadas o en "Contrato".
<code>valor_cotizado</code>	Suma del importe de todas sus oportunidades de proyectos.
<code>potencia_instalada_kwp</code>	Suma de la "Potencia (kWp)" de las oportunidades cerradas.
<code>fecha_calificado</code>	Cuándo pasó a calificada (para la racha).
<code>clientify_updated_at</code> , <code>updated_at</code>	Última actualización desde Clientify y en la base.

avance_conflictos — cambios de Clientify sobre valores ya definitivos

Columna	Qué representa
<code>id</code>	Llave del conflicto.
<code>empresa_id</code>	Empresa afectada.
<code>variable</code>	Qué variable cambió (p. ej. <code>calificado</code>).

Columna	Qué representa
valor_hub, valor_clientify	Valor definitivo en el Hub y nuevo valor en Clientify.
detectado_at	Cuándo se detectó.
resuelto_at, resuelto_por, nota	Cuándo, quién y cómo lo resolvió un admin.

7.5 Puntos

reglas_puntos — catálogo de motivos

Columna	Qué representa
motivo	Código de la regla (p. ej. empresa_calificada).
tipo	ganado, perdido o redimido (vacío si puede ser ambos, como ajuste_admin).
puntos	Valor fijo; vacío si varía (módulos, ajustes, canjes).
descripcion	Texto que ve el aliado en el historial.

movimientos_puntos — libro mayor (única fuente de verdad de los puntos)

Columna	Qué representa
id	Llave del movimiento.
aliado_id	Aliado afectado.
tipo	ganado, perdido o redimido.
puntos	Valor nominal de la regla.
puntos_aplicados	Lo que realmente movió el saldo (una pérdida con saldo bajo aplica menos).
motivo	Regla aplicada (llave foránea a reglas_puntos).
vinculo, vinculo_id	Qué lo originó: empresas, eventos, modulos_completados, racha, canjes o ajuste_admin, y qué fila.
clave_unica	Evita duplicados (p. ej. empresa:{id}:calificado).
fecha	Cuándo se otorgó (define semanas, meses y la ventana de 6 meses).
creado_por	sistema, webhook_clientify, canjes_api o admin:{id}.
nota	Detalle (justificación del ajuste, nombre de la recompensa...).
secuencia	Orden exacto cuando dos movimientos tienen la misma hora.

movimientos_retenidos — puntos en espera mientras la cuenta no está activa

Mismas columnas que el libro mayor, más: fecha_original (cuándo ocurrió), estado_aliado (por qué se retuvo) y retenido_at. Al activarse la cuenta pasan al libro mayor en su orden original y se borran de aquí.

7.6 Academy, eventos y canjes

modulos — catálogo de cursos

Columna	Qué representa
id	Llave.

Columna	Qué representa
<code>codigo</code>	ID del curso en la Academy del Hub (<code>c11</code> , <code>c12</code> ...).
<code>nombre</code> , <code>orden</code> , <code>activo</code>	Nombre, orden de aparición y si está disponible.
<code>puntos</code>	Valor del módulo (0-20; 0 = sin puntos).
<code>created_at</code> , <code>updated_at</code>	Fechas.

modulos_completados

Columna	Qué representa
<code>id</code>	Llave.
<code>aliado_id</code> , <code>modulo_id</code>	Quién completó qué (único por par).
<code>fecha_completado</code>	Cuándo lo terminó.
<code>puntos</code>	Valor del módulo en ese momento.
<code>recompensa_estado</code>	<code>pendiente</code> (espera cupo en el tope mensual), <code>otorgada</code> o <code>no_aplica</code> .
<code>fecha_otorgada</code>	Cuándo se dieron los puntos.

eventos — eventos aliados

Columna	Qué representa
<code>id</code> , <code>aliado_id</code>	Llave y aliado que lo reporta.
<code>nombre_evento</code> , <code>tipo_evento</code> , <code>fecha</code> , <code>descripcion</code>	Datos del evento.
<code>genera_involucrada</code> , <code>registro_asistentes</code> , <code>empresas_perfil_count</code>	Las condiciones para los +100.
<code>registro_asistentes_path</code>	Archivo en el bucket privado <code>eventos</code> .
<code>estado</code>	<code>pendiente</code> , <code>validado</code> o <code>rechazado</code> .
<code>validado_por</code> , <code>validado_at</code> , <code>revision_nota</code>	Admin que lo revisó, cuándo y su nota o motivo de rechazo.
<code>created_at</code> , <code>updated_at</code>	Fechas.

recompensas — catálogo de canjes

Columna	Qué representa
<code>id</code>	Llave.
<code>codigo</code>	Código que usa el proveedor para canjear. Inmutable.
<code>nombre</code> , <code>descripcion</code> , <code>categoria</code>	Lo que ve el aliado.
<code>puntos</code>	Costo en Puntos Sol.
<code>nivel_minimo</code>	Nivel necesario para canjearla.
<code>proveedor</code>	Si tiene valor, solo ese proveedor puede canjearla.
<code>activa</code>	Si se puede canjear hoy.
<code>created_at</code> , <code>updated_at</code>	Fechas.

canjes

Columna	Qué representa
id, aliado_id	Llave y aliado.
recompensa_id, recompensa	Recompensa del catálogo y su nombre al momento del canje.
puntos, nivel_requerido	Puntos descontados y nivel exigido al momento del canje.
proveedor, referencia_externa	Quién lo registró y su número de operación (único por proveedor).
fecha	Cuándo.
estado	confirmado o anulado.
anulado_at, anulado_por, anulacion_motivo	Datos de la anulación.

7.7 Integración y auditoría

webhook_eventos — lo que llegó de Clientify

Columna	Qué representa
id	Llave.
fuelle	Hoy solo clientify.
payload	Contenido recibido. Se vacía al procesarlo si no es del programa, y a los 90 días en todos los casos.
recibido_at, procesado_at	Cuándo llegó y cuándo se procesó.
error	Error o avisos del proceso (Status desconocido, conflictos...).
entidad, entidad_id, accion	Qué contacto u oportunidad y qué pasó (created, saved ...).

clientifyColaEntidades — cola de trabajo del flujo C

Columna	Qué representa
entidad, entidad_id	Contacto u oportunidad de Clientify (llave primaria compuesta: una sola fila por entidad).
origen	Webhook o conciliación.
programado_at	Cuándo procesarla (permite esperas y reintentos).
intentos, error	Reintentos y último error.
actualizado_at	Última actualización.

acciones_admin — auditoría del panel (solo inserción)

Columna	Qué representa
id	Llave.
admin_id, admin_codigo	Admin que actuó (el código se conserva aunque se borre la cuenta).

Columna	Qué representa
accion	Qué hizo (aprobar, rechazar, suspender, ajustar, validar evento, anular canje, guardar recompensa...).
aliado_id, aliado_codigo	Aliado afectado, si aplica.
objetivo	Otro objeto afectado (evento:{id}, canje:{id}, recompensa:{codigo} ...).
detalle	Motivo, puntos, valores anteriores, etc.
created_at	Cuándo.

7.8 Vistas, archivos y funciones

- **Vistas del aliado** (cada una devuelve solo lo del usuario de la sesión): `v_aliado_dashboard`, `v_mis_movimientos`, `v_mis_referidos`, `v_mis_modulos`, `v_mis_eventos`, `v_mis_canjes` y `v_recompensas`. **Ninguna expone aliados.id**.
- **Vistas del admin** (vacías para quien no es admin): `v_admin_resumen`, `v_admin_aliados`, `v_admin_movimientos`, `v_admin_eventos`, `v_admin_conflictos`, `v_admin_canjes`, `v_admin_recompensas` y `v_admin_acciones`.
- **Buckets privados de Storage:**
 - `facturas`: PDF, JPG o PNG de hasta 10 MB.
 - `eventos`: PDF, imagen, Excel o CSV de hasta 10 MB.
 - El navegador sube con una URL firmada de un solo uso y solo el servidor lee.
- **Funciones principales:**
 - `handle_new_aliado`: registro.
 - `registrar_oportunidad`: Nueva oportunidad.
 - `aplicar_avance_clientify`: flujo C.
 - `completar_modulo` y `otorgar_modulos_pendientes`: Academy.
 - `registrar_evento`: eventos.
 - `registrar_canje` y `consultar_canjes`: canjes.
 - `admin_*`: acciones del panel.
 - `recalcular_aliado`, `calcular_nivel`, `calcular_puntos_nivel` y `calcular_puntos_disponibles`: cálculos.

8. Seguridad y variables de entorno

Principios:

- **RLS en las 18 tablas.** Un aliado solo puede **leer** sus propias filas. Las escrituras las hacen solo el servidor (clave secreta) o funciones controladas.
- **Nunca se muestra el id interno.** Hacia afuera (Clientify, proveedores, pantalla) solo sale el `codigo_aliado`.
- **Todo endpoint verifica la sesión y exige `estado = 'activo'`**. Las acciones de admin se verifican dos veces: en el endpoint y otra vez en la base.
- **Límites anti-abuso:** 20 referidos por hora, 5 eventos por día y 30 canjes por hora por aliado.
- **Las claves secretas nunca van en el código ni en los commits.**

Variables de entorno en Vercel (cada una con valor de `prod` para *Production* y de `dev` para *Preview/Development*):

Variable	Qué es	¿Llega al navegador?
<code>SUPABASE_URL</code>	Dirección del proyecto de Supabase.	Sí, por <code>/api/config</code> .
<code>SUPABASE_PUBLISHABLE_KEY</code>	Clave pública (la seguridad la da RLS).	Sí, por <code>/api/config</code> .
<code>SUPABASE_SECRET_KEY</code>	Clave secreta de servidor: salta RLS.	Nunca.
<code>CLIENTIFY_API_KEY</code>	Acceso a la API de Clientify.	Nunca.
<code>CLIENTIFY_WEBHOOK_SECRET</code>	Token que Clientify y n8n envían al webhook.	Nunca.
<code>CRON_SECRET</code>	Protege <code>/api/cron/*</code> (debe ser igual al <code>cron_secret</code> del Vault).	Nunca.
<code>CANJES_API_KEYS</code>	<code>proveedor:key,proveedor:key</code> (keys de 24+ caracteres).	Nunca.

Funciones de `/api` (12):

Función	Uso
<code>config</code>	Configuración pública.
<code>oportunidades/factura</code> y <code>oportunidades</code>	Nueva oportunidad.
<code>modulos</code>	Academy.
<code>eventos</code>	Reportar eventos.
<code>admin</code>	Acciones del panel.
<code>canjes</code>	Proveedores.
<code>webhooks/clientify</code>	Webhook de Clientify.
<code>cron/clientify</code> , <code>cron/clientify-aliados</code> (alias), <code>cron/clientify-conciliacion</code> y <code>cron/clientify-diagnostico</code>	Tareas programadas y diagnóstico.

9. Decisiones tomadas

Resumen agrupado. El detalle y la sección de cada una están en `CLAUDE.md` §13.

Registro y cuentas - Correo confirmado obligatorio y aprobación de GEENERA antes de entrar. Una solicitud rechazada queda en `rechazado` y se puede aprobar después. - Celular internacional con selector de país; regla colombiana (10 dígitos que empiezan por 3) para +57. - "¿Cómo llegas a las empresas?" es obligatoria para EMI, Linker y Cliente Embajador. La regional es opcional. - Iniciales del código: se ignoran las partículas y se usan máximo 4 letras. - Los Términos aplican a todos los tipos de aliado. La Política de datos debe incluir la transferencia internacional. - El primer admin es `c.lizarazo@geenera.com`. Los roles se asignan por SQL: el panel no cambia roles. - Las cuentas de admin no se sincronizan con Clientify.

Clientify - El aliado se crea en Clientify al aprobarse, no al registrarse. Etiqueta "aliado del sol hub" para Cliente Embajador y "aliados del sol" para los demás. - Las variables se derivan del Status del contacto, la fase de la oportunidad y las etiquetas. Clientify no tiene campos sí/no/revisión. - Cuentan todos los embudos de proyectos, por el nombre de la fase ("Diseño", "Presentación de oferta", "Contrato"). Con varias oportunidades se usa la más avanzada. - "0. lead perdido" = revisión. La fase "7. Interesado No ahora" cuenta como propuesta. - Información falsa = etiquetas "fraude", "no existe" o "información de contacto errónea". - Valor cotizado = suma del importe de todas las oportunidades. Pipeline originado = suma de las cerradas. - La factura se adjunta a la **empresa** en Clientify, y la empresa se crea siempre. - Un contacto se refiere una sola vez (gana el primer aliado) y no se admite el autorreferido. - El webhook de contactos sigue en n8n; al Hub llega el de oportunidades, más una conciliación horaria. - Los eventos de webhook ajenos al programa se borran al procesarlos. El resto se vacía a los 90 días. - Un cambio de Clientify sobre un valor definitivo queda como conflicto para un admin.

Puntos, calidad y niveles - "Información disponible" se eliminó. Referido perfecto = +20. - Fuera del perfil (-15) se suma a no calificado (-10). - Piso en 0 y sin memoria. Los puntos de un aliado no activo se retienen. - La calidad usa siempre la fórmula ponderada y excluye las empresas en revisión. El *lead scoring* nunca se usa. - Nivel = el menor entre el nivel por puntos y el nivel por calidad. El Hub no muestra el requisito de calidad. - Racha: cuenta la fecha de calificación; se reinicia el lunes después de completarla; lo retenido no cuenta. - Módulos: cada uno vale lo que diga el catálogo; tope de 20 puntos al mes, sin partir módulos y en orden de llegada. - Máximo 20 referidos por hora.

Hub, panel y canjes - Financieros y Agremiaciones también ven sus puntos y su nivel. Agremiaciones agrupan por la ciudad de la empresa referida. - Las secciones sin datos reales muestran datos de demostración con la etiqueta "Demostración". - Academy: en la base solo se guarda el curso completado; el avance por lección vive en el navegador. - El panel es una página separada (`admin.html`). Los eventos los reporta el aliado y los valida un admin. - El catálogo de recompensas se administra desde el panel. El proveedor puede consultar el nivel y el saldo por código, sin datos personales. Un admin puede anular un canje: se devuelve el saldo, no los puntos de nivel.

10. Próximos pasos para poner en marcha el Hub

Los pasos están en orden. Cada uno dice **qué hacer** y **por qué**.

Paso 1 — Fusionar las fases en `main`

Qué: revisar y fusionar el *pull request* de `fase-10-canjes` hacia `main`, que contiene las fases 1 a 10. **Por qué:** hoy `main` no tiene nada de lo construido. Vercel publica Producción desde `main`. **Antes de fusionar**, confirma que las variables de *Production* apuntan a `prod` (paso 5), o que la protección de despliegues evita que alguien use la URL de producción mientras se prepara.

Paso 2 — Decidir los planes (\$4)

Qué: pasar Vercel a Pro. Crear una organización de Supabase en Pro para `aliados-prod` y dejar `aliados-dev` en una organización gratuita. **Por qué:** Vercel Hobby no permite uso comercial. Supabase Free pausa el proyecto tras 7 días de poca actividad y no ofrece copias descargables.

Paso 3 — Crear el esquema en `aliados-prod`

Qué: aplicar en orden las 23 migraciones de `supabase/migrations/` al proyecto de producción. Opciones: - **Recomendada:** Supabase CLI desde un computador del equipo: 1. `npx supabase login` 2. `npx supabase link --project-ref pysyxcrybayhrescplc` 3. `npx supabase db push` -

Alternativa: la integración GitHub → Supabase, para que cada fusión en `main` aplique las migraciones.

Luego verificar en el panel de Supabase: - que las extensiones `pg_cron`, `pg_net` y Vault estén activas; - que se vean las 18 tablas y los 6 cron jobs; - que el *Security Advisor* no muestre alertas nuevas.

Por qué: las migraciones son la receta exacta de la base, probada en `dev`. Crear las tablas a mano en `prod` produciría diferencias difíciles de detectar. Claude no toca `prod` directamente, por decisión del equipo.

Paso 4 — Configurar la autenticación de `prod`

Qué: en Supabase → Authentication: 1. **URL Configuration:** *Site URL* = el dominio oficial del Hub; *Redirect URLs* = el mismo dominio. 2. **Providers → Email:** confirmación de correo **activada**. 3. **SMTP propio:** conectar un servicio de correo (Resend, Brevo, Amazon SES, Postmark...) con un remitente del dominio de GEENERA (p. ej. `no-reply@geenera.com`), con SPF, DKIM y DMARC configurados. 4. **Rate limits:** subir el límite de correos por hora según el lanzamiento (con SMTP propio arranca en 30 por hora). 5. Revisar y traducir las **plantillas de correo** (confirmación y recuperación de contraseña). 6. Activar la **protección de contraseñas filtradas** (hoy es un aviso del *Security Advisor*).

Por qué: sin SMTP propio, Supabase solo envía correos a los miembros del equipo del proyecto y con un límite muy bajo. Los aliados reales no recibirían el correo de confirmación y no podrían entrar.

Paso 5 — Variables de entorno en Vercel

Qué: en Vercel → Settings → Environment Variables, para *Production*: - `SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY` y `SUPABASE_SECRET_KEY` de `aliados-prod`; - `CLIENTIFY_API_KEY`; - un `CLIENTIFY_WEBHOOK_SECRET` **nuevo**; - un `CRON_SECRET` **nuevo**; - `CANJES_API_KEYS` cuando exista el proveedor.

Para *Preview* y *Development*: los de `aliados-dev`. Después, volver a desplegar Producción.

Por qué: es lo que separa el mundo de pruebas del real (§3.2). Los secretos de producción deben ser distintos de los de pruebas, y los que se compartieron en conversaciones deben rotarse.

Paso 6 — Dominio y protección

Qué: - Asignar el dominio oficial al proyecto en Vercel. - Dejar Producción **pública** y los Preview **protegidos**. - Si se protege algo que el cron necesita llamar, generar el *Protection Bypass for Automation*.

Por qué: los aliados entran por el dominio oficial, y los Preview no deben ser visibles al público.

Paso 7 — Secretos del Vault en `prod`

Qué: en Supabase `prod` → Vault, crear: - `clientify_sync_url` = `https://<dominio>/api/cron/clientify`; - `cron_secret` = el mismo valor de `CRON_SECRET` de Production; - `vercel_bypass_secret`, solo si Producción está protegida.

Por qué: los cron de `pg_cron` usan estos valores para llamar a Vercel. Sin ellos, la sincronización con Clientify y el procesamiento de webhooks no ocurren.

Paso 8 — Webhooks de Clientify y n8n

Qué: 1. En Clientify, cambiar el webhook de **oportunidades** a `https://<dominio>/api/webhooks/clientify?token=<CLIENTIFY_WEBHOOK_SECRET de producción>`. Hoy apunta a un Preview de una rama. 2. Mantener un webhook separado para pruebas que apunte al Preview, con el secreto de `dev`, si se sigue probando. 3. En n8n, agregar la rama que reenvía al Hub los contactos con etiqueta de aliado (cuerpo mínimo, encabezado `x-webhook-token`; ver `CLAUDE.md` §8).

Por qué: sin el webhook, el Hub no se entera a tiempo de los avances comerciales (se enteraría solo por la conciliación horaria). Sin la rama de n8n, un lead del formulario público que nunca tenga oportunidad no se descubre.

Paso 9 — Documentos legales

Qué: subir a `assets/legal/` la nueva Política de Tratamiento de Datos (con la transferencia internacional), actualizar `JOIN_CONFIG.legal` en la página y la función de versión vigente con una migración. Validar todo con el asesor legal. **Por qué:** la Ley 1581 exige autorización informada. La base guarda qué versión aceptó cada aliado, así que publicar la versión definitiva **antes** de abrir el registro evita tener aliados con una versión anterior.

Paso 10 — Primer admin en producción

Qué: 1. La persona se registra en el Hub de Producción y confirma su correo. 2. Se ejecuta en el *SQL Editor* de `prod`: `sql update public.aliados set rol = 'admin', estado = 'activo', aprobado_at = coalesce(aprobado_at, now()) where lower(correo) = 'c.lizarazo@geenera.com'`; 3. Se comprueba el acceso a `https://<dominio>/admin.html`.

Por qué: el panel no asigna roles, por seguridad. La cuenta de admin queda excluida de Clientify automáticamente.

Paso 11 — Prueba de punta a punta con un aliado piloto

Qué: con una o dos personas de confianza: 1. Registro y confirmación de correo. 2. Aprobación desde el panel y verificación de que aparece en Clientify con su `ID_aliado`. 3. Una Nueva oportunidad con factura y verificación de que llega a Clientify. 4. Avance de esa oportunidad en Clientify (calificar, Diseño) y verificación de los puntos en el Hub (hasta 1 hora de espera). 5. Completar un curso de la Academy y reportar un evento.

Al terminar, limpiar en Clientify los datos de prueba, o marcarlos, porque en Producción no se agrega `PRUEBA HUB`.

Por qué: es la única forma de confirmar que dominio, correo, variables, Vault, cron y webhooks están bien conectados en producción.

Paso 12 — Abrir el registro y operar

Qué: - Comunicar el enlace "Quiero ser aliado". - Revisar el panel **a diario**: solicitudes, eventos por revisar y conflictos. - Crear el catálogo de recompensas cuando se definan los beneficios y el proveedor, y darle al proveedor su API key.

Por qué: los aliados no entran hasta ser aprobados, así que una solicitud sin revisar es un aliado que se enfría.

Operación continua (checklist)

- ☐ **Diario:** revisar solicitudes, eventos y conflictos en el panel.
- ☐ **Semanal:** revisar en `v_admin_aliados` los aliados con `clientify_sync_estado = 'error'`, y los logs de funciones en Vercel.
- ☐ **Mensual:** revisar el uso y los costos en Vercel y Supabase, y el *Security Advisor* de Supabase.
- ☐ **Con Supabase Free:** exportar la base periódicamente y entrar al panel para evitar la pausa.
- ☐ **En cada cambio:** rama nueva → Preview → pruebas → *pull request* → migración en `prod` → fusión.

11. Pendientes y preguntas abiertas

#	Tema	Estado
1	Sistema externo de canjes: quién es el proveedor y qué recompensas habrá.	El Hub ya está listo (API y catálogo). Falta definir el proveedor, generar su key y cargar el catálogo.
2	Rama de n8n para reenviar los leads del formulario público.	Plan definido (CLAUDE.md §8); falta implementarla en n8n.
3	Etiquetas de los flujos A y B: ¿el contacto del aliado debe llevar "aliados del sol"/"aliado del sol hub" si esas etiquetas disparan automatizaciones de "nuevo lead"?	Por confirmar con el equipo comercial.
4	Nueva versión de la Política de Tratamiento de Datos (transferencia internacional, finalidades y canal de reclamos).	Pendiente de redacción legal.
5	Política de beneficios (se incluirá en los Términos).	Pendiente.
6	Webhook de oportunidades en Producción y rotación de los secretos compartidos en chat.	Pendiente (pasos 5 y 8).
7	Secciones del Hub con datos de demostración (series por mes, pronóstico de desembolsos, comisiones).	Se mantienen con la etiqueta "Demostración" hasta tener datos reales.
8	Imágenes y logos de las recompensas reales.	El catálogo aún no guarda imágenes.

12. Glosario

Término	Significado
Aliado	Persona u organización registrada que refiere empresas.
Referido / lead	Empresa referida por un aliado.
codigo_aliado / ID_aliado	Código público del aliado; en Clientify es el campo <code>ID_aliado</code> .
Puntos Sol	Puntos del programa.
Libro mayor	La tabla <code>movimientos_puntos</code> : registro de todos los movimientos, solo con inserciones.
Clave única	Texto que identifica un evento de puntos para que no se otorgue dos veces.
Migración	Archivo SQL versionado que cambia la estructura de la base.
RLS (Row Level Security)	Reglas de la base que deciden qué filas puede leer cada usuario.
Publishable key / Secret key	Clave pública de Supabase (va al navegador) y clave secreta (solo servidor, salta RLS).
Preview / Production	Despliegue de prueba de una rama (usa <code>dev</code>) y despliegue oficial desde <code>main</code> (usa <code>prod</code>).
Webhook	Aviso automático que un sistema (Clientify) envía a otro (el Hub) cuando algo cambia.
Conciliación	Revisión periódica que vuelve a consultar Clientify para cubrir avisos perdidos.
Cron	Tarea programada que se ejecuta sola cada cierto tiempo.
Vault	Almacén cifrado de secretos dentro de Supabase.
Pull request (PR)	Solicitud para fusionar una rama en <code>main</code> , donde se revisan los cambios.
pgTAP	Herramienta de pruebas automáticas para la base de datos.
URL firmada	Enlace temporal, de un solo uso, para subir o ver un archivo privado.
Flujo A / B / C	Aliado aprobado → Clientify; Nueva oportunidad → Clientify; Clientify → Hub.